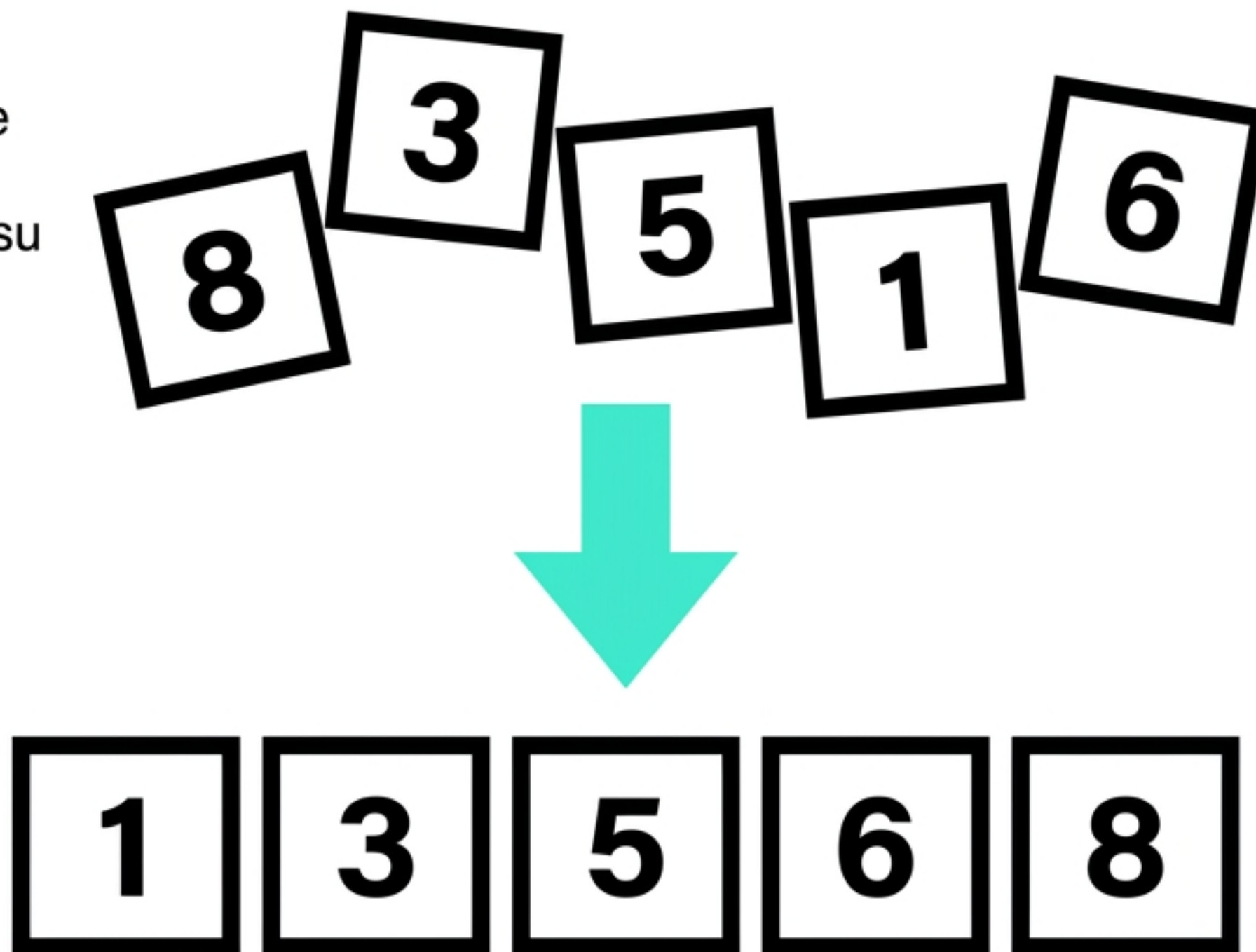


Métodos de Ordenamiento

Algoritmos, Eficiencia e Implementación en Python

¿Qué es ordenar?

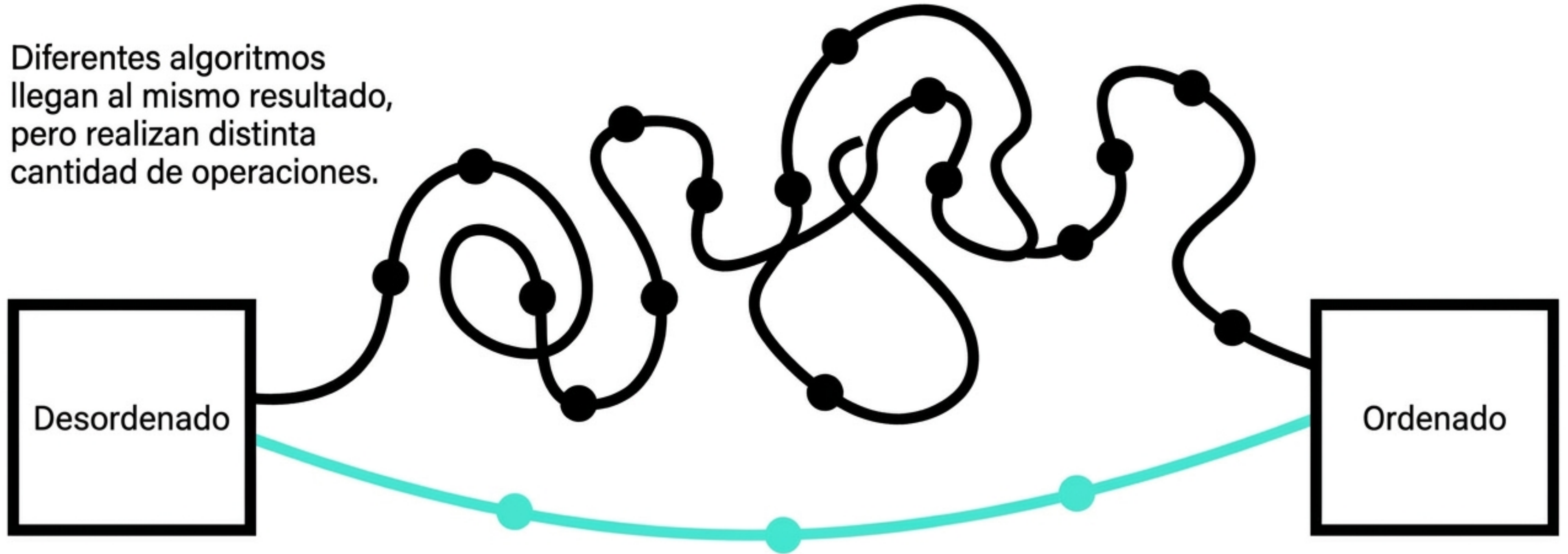
Disponer un conjunto de datos en un orden específico para facilitar su búsqueda y análisis.



Ascendente (1→9) | Descendente (9→1) | Alfabético (A→Z)

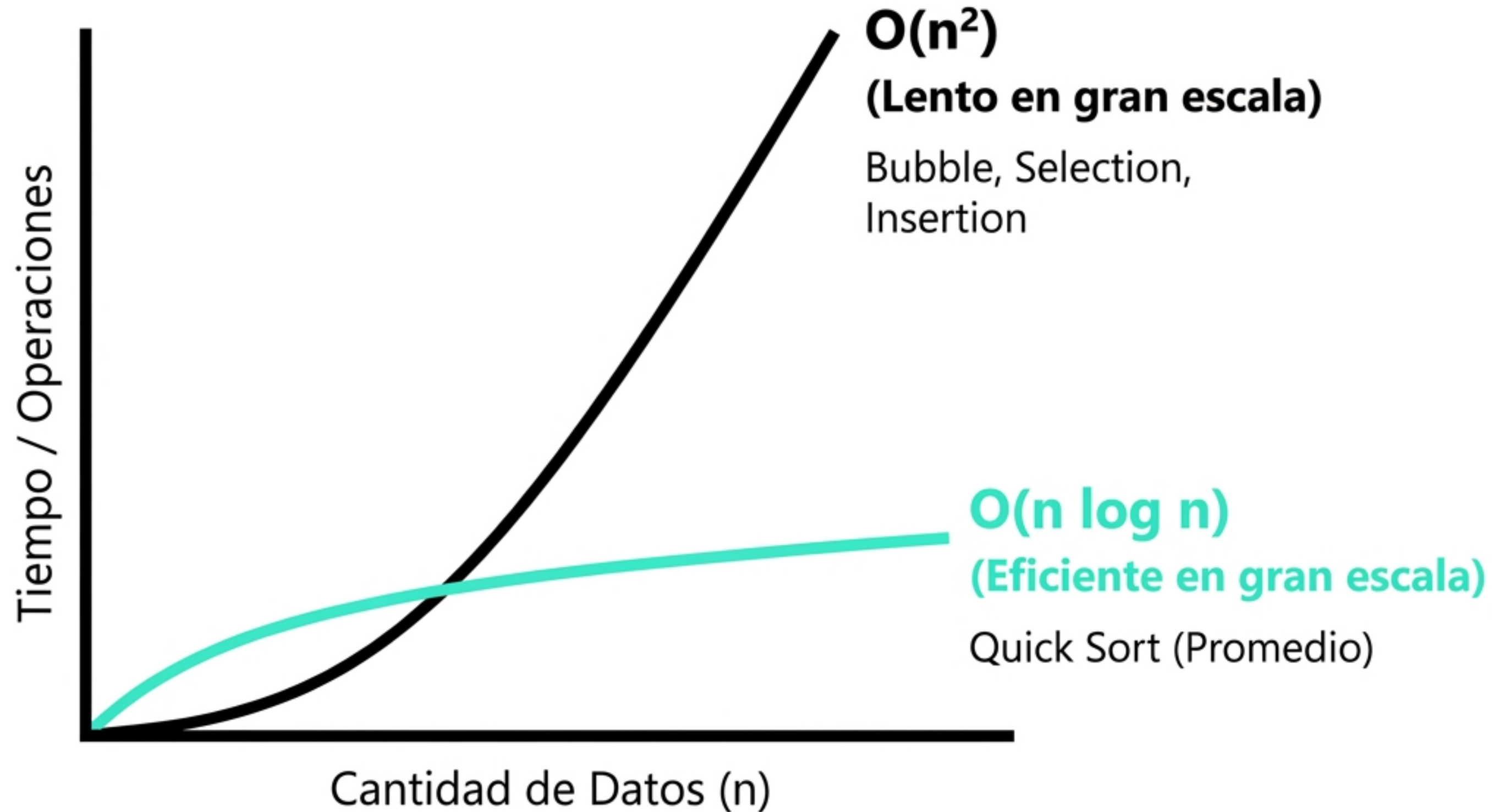
¿Por qué hay distintos métodos?

Diferentes algoritmos
llegan al mismo resultado,
pero realizan distinta
cantidad de operaciones.

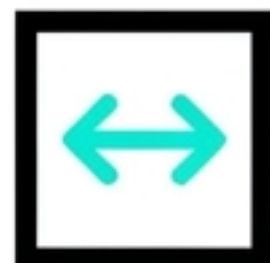


La eficiencia mide la calidad del algoritmo respecto al tamaño de los datos. Distinto camino, mismo resultado.

Big O aplicado al Ordenamiento



Métodos que veremos



Bubble Sort

Compara vecinos.



Selection Sort

Busca el menor.



Insertion Sort

Inserta en posición correcta.



Shell Sort

Compara usando saltos (gaps).

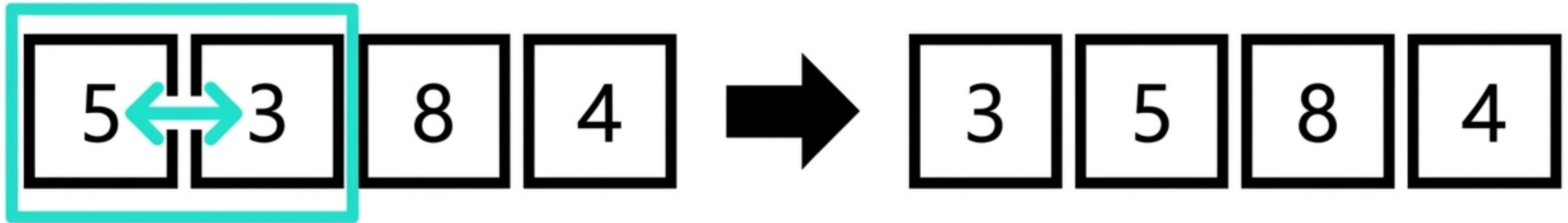


Quick Sort

Divide usando pivote.

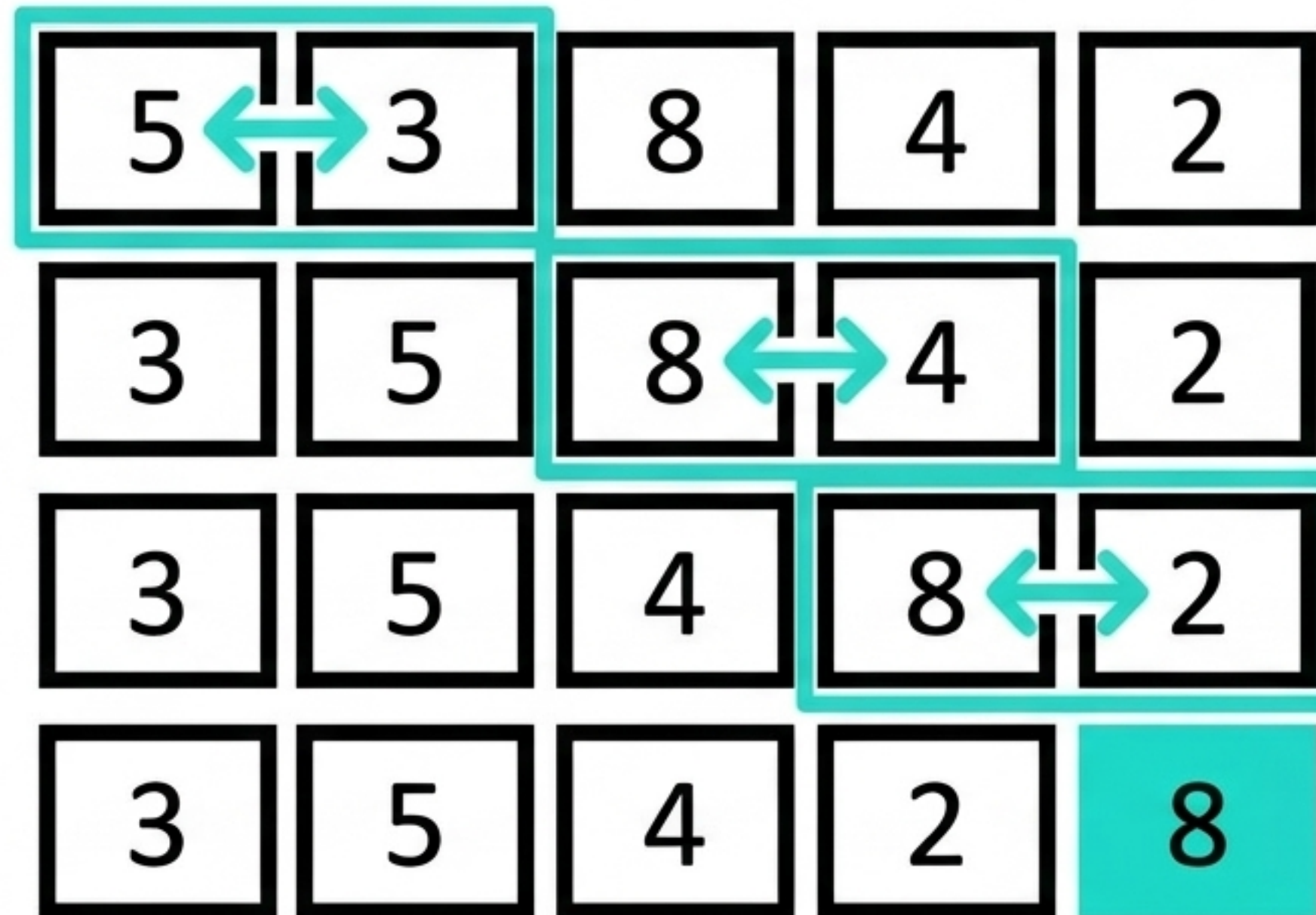
Bubble Sort: Concepto

Comparación directa entre elementos vecinos. Si están en el orden incorrecto, se intercambian.



Bubble Sort: Proceso

En cada iteración, el elemento mayor “burbujea” hacia la última posición disponible.



Bubble Sort en Python

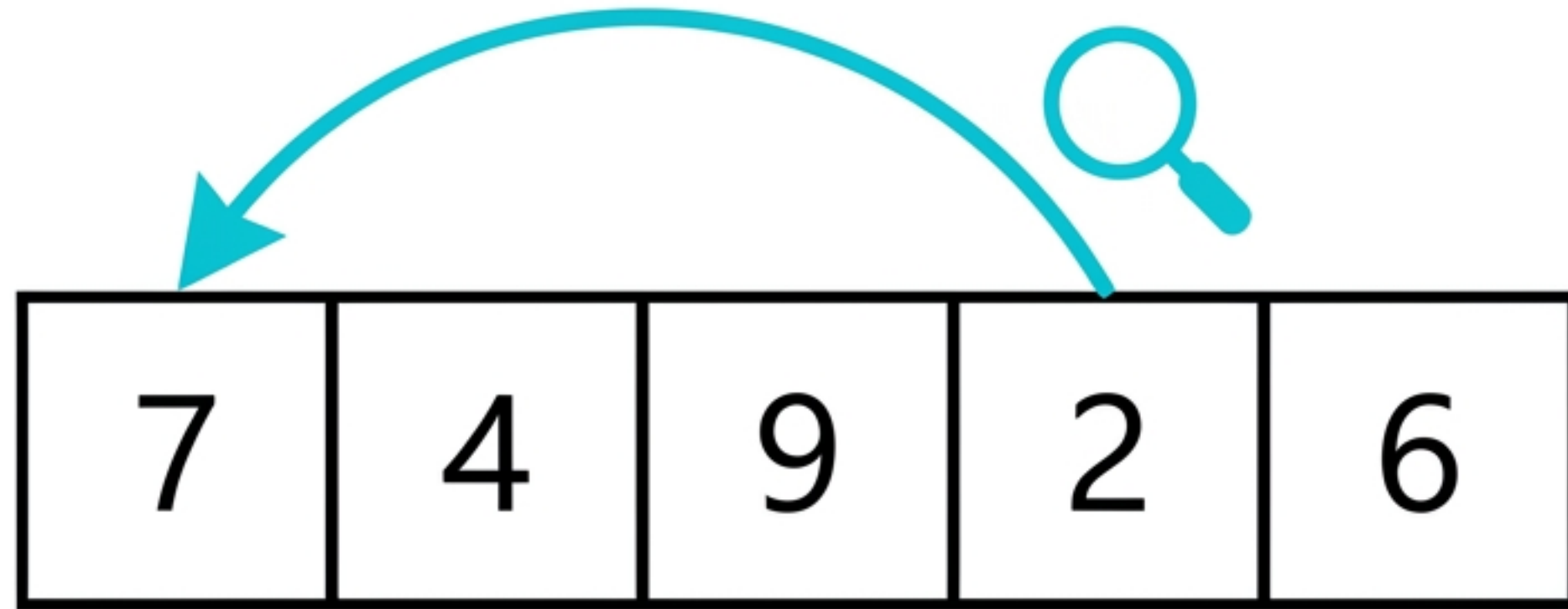
Complejidad: $O(n^2)$

```
def bubble_sort(lista):  
    n = len(lista)  
    for i in range(n):  
        for j in range(0, n - i - 1):  
            if lista[j] > lista[j + 1]:  
                lista[j], lista[j + 1] = lista[j + 1], lista[j]  
    return lista
```

Entrada: [5, 3, 8, 4, 2] → Salida: [2, 3, 4, 5, 8]

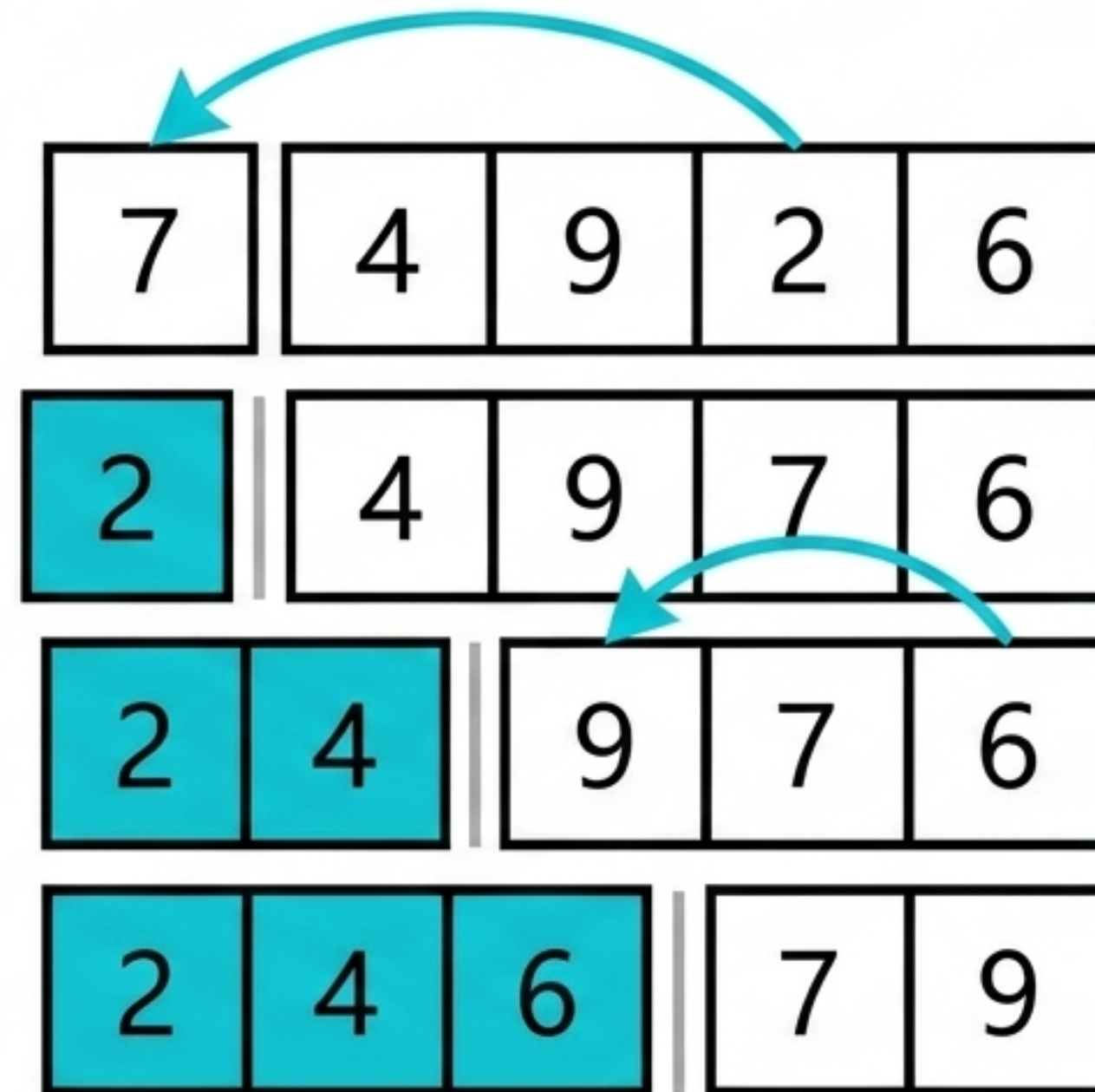
Selection Sort: Concepto

Busca el menor elemento de la lista y lo coloca en la primera posición disponible.



Selection Sort: Proceso

La zona ordenada crece un elemento por paso tras encontrar el mínimo absoluto del resto.



Selection Sort en Python

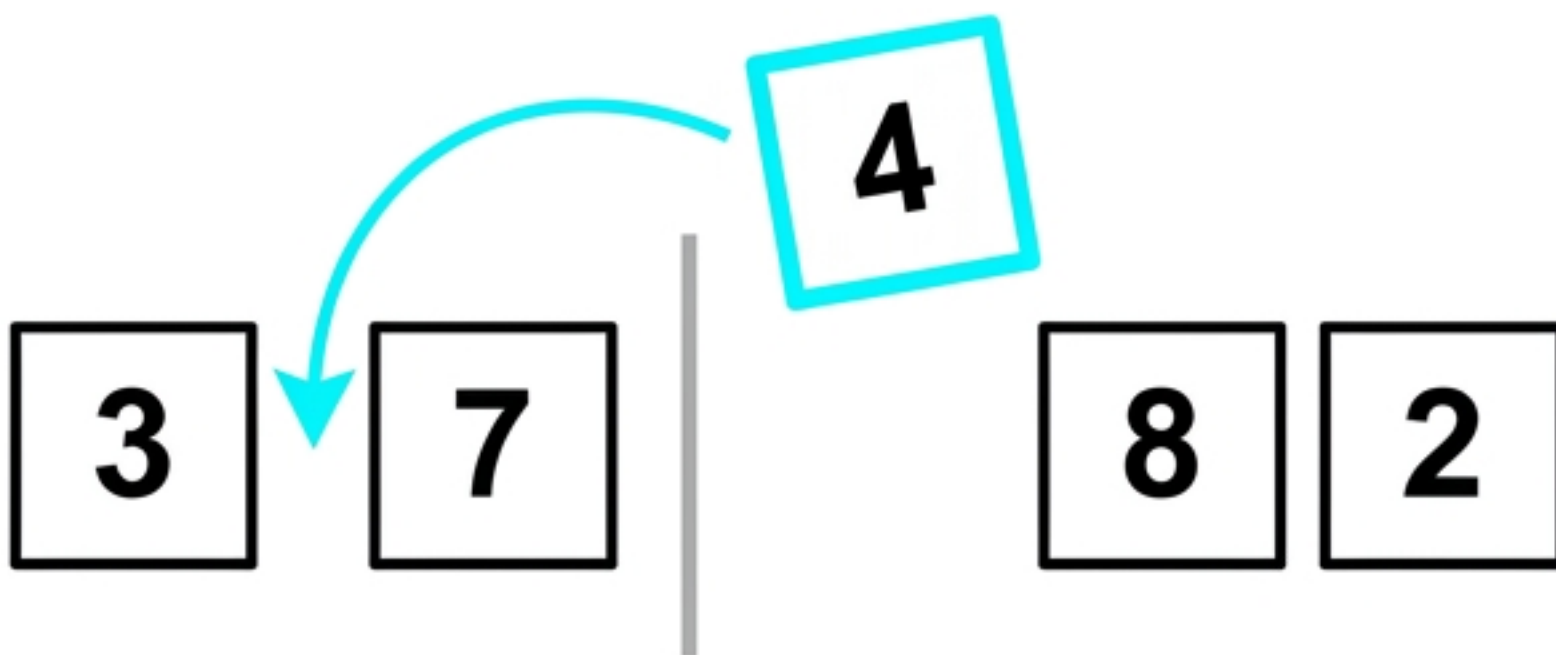
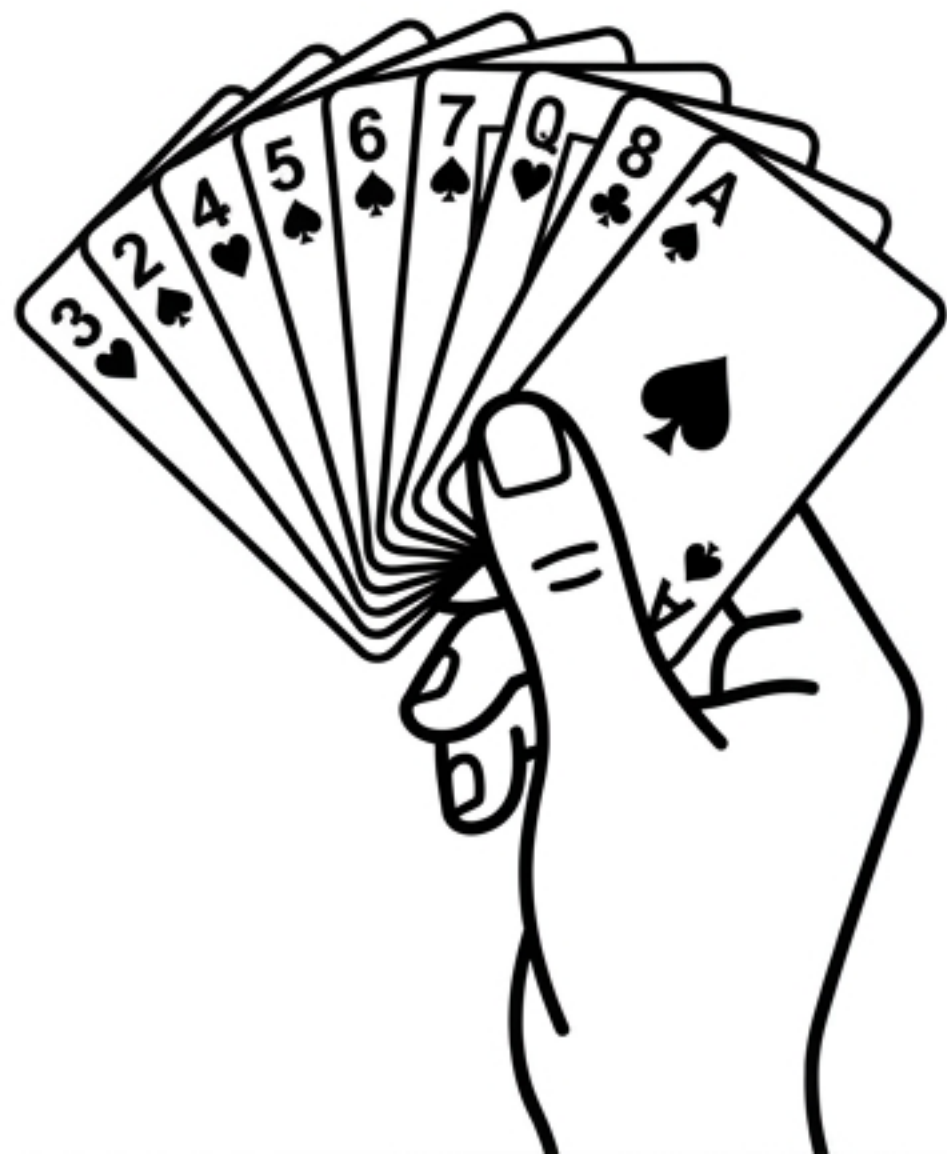
Complejidad: $O(n^2)$

Realiza muchas comparaciones, pero pocos intercambios.

```
def selection_sort(lista):  
    for i in range(len(lista)):  
        min_idx = i  
        for j in range(i + 1, len(lista)):  
            if lista[j] < lista[min_idx]:  
                if lista[j] < lista[min_idx]:  
                    min_idx = j  
        lista[i], lista[min_idx] = lista[min_idx], lista[i]  
    return lista
```

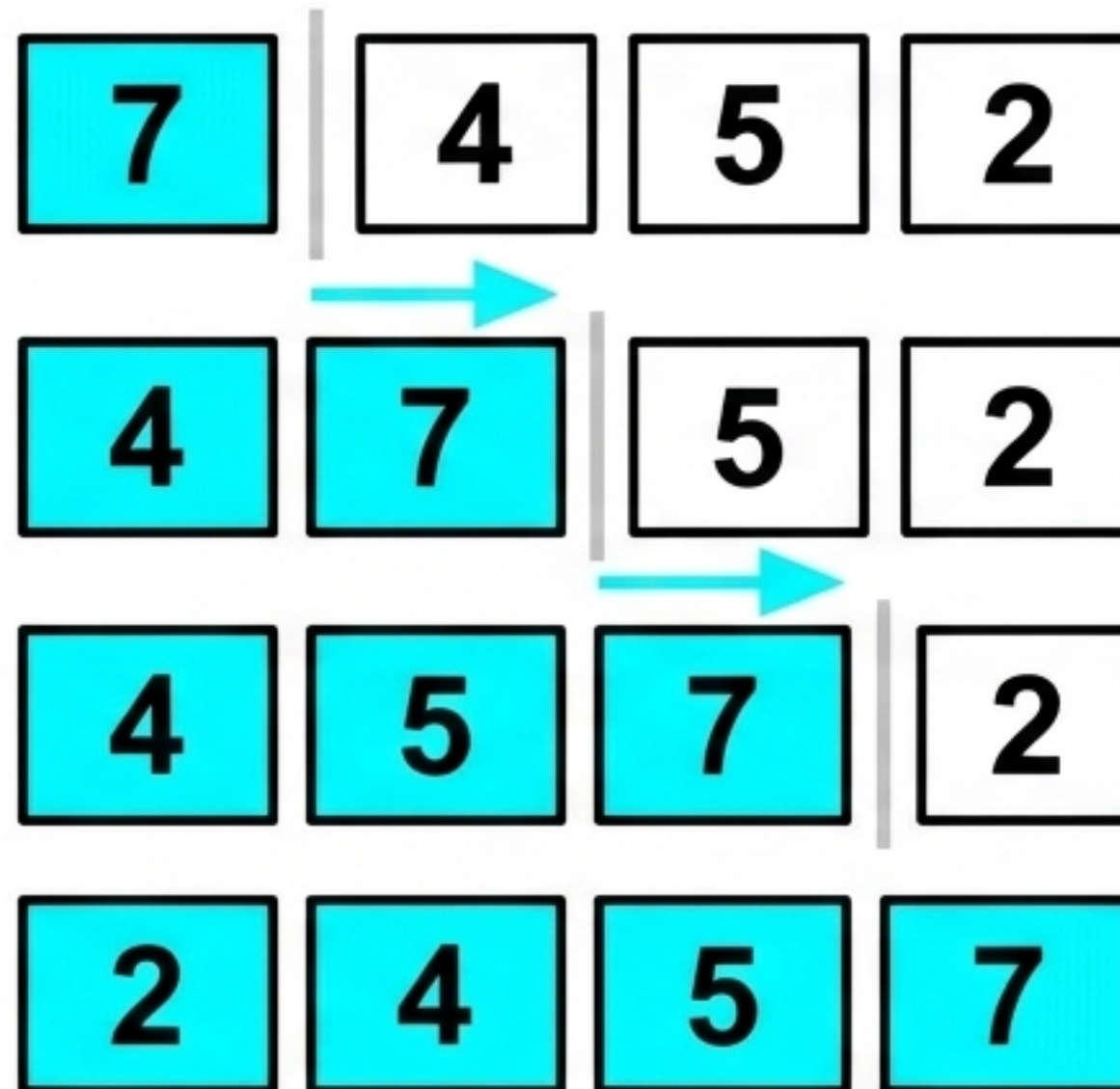
Insertion Sort: Concepto

Como ordenar cartas en tu mano. Toma un elemento de la parte sin ordenar y lo inserta en su posición correcta en la parte ordenada.



Insertion Sort: Proceso

Los elementos mayores se desplazan hacia la derecha para hacer espacio al nuevo elemento.



Insertion Sort en Python

Mejor: $O(n)$

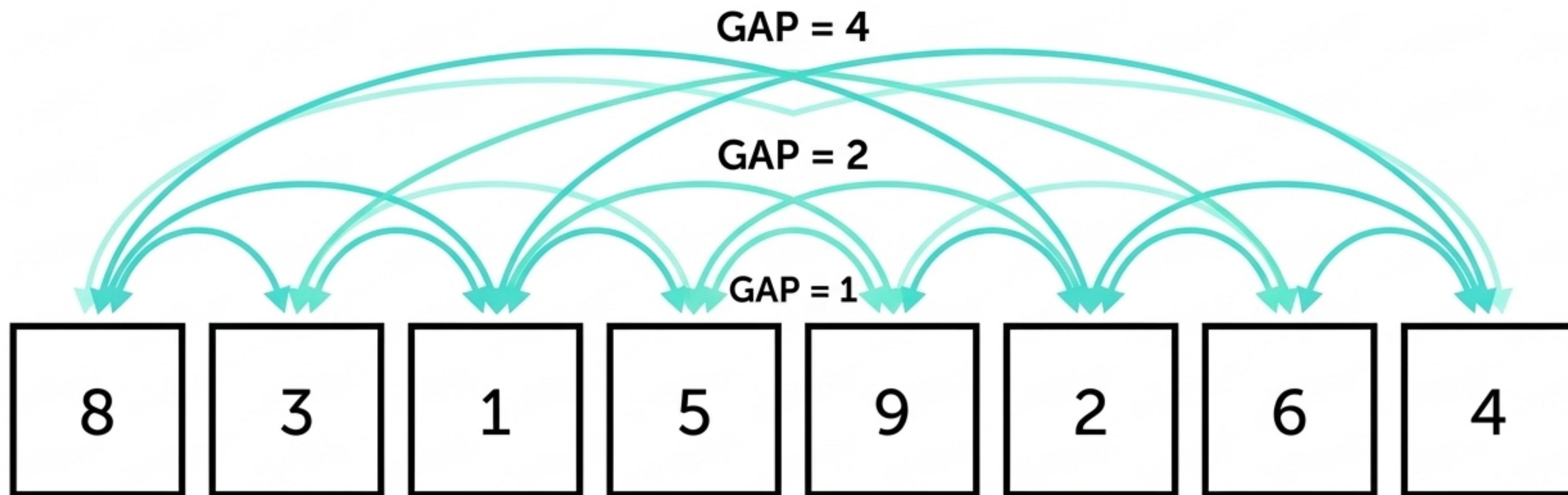
Peor: $O(n^2)$

Excelente y muy rápido para listas pequeñas o que ya están casi ordenadas.

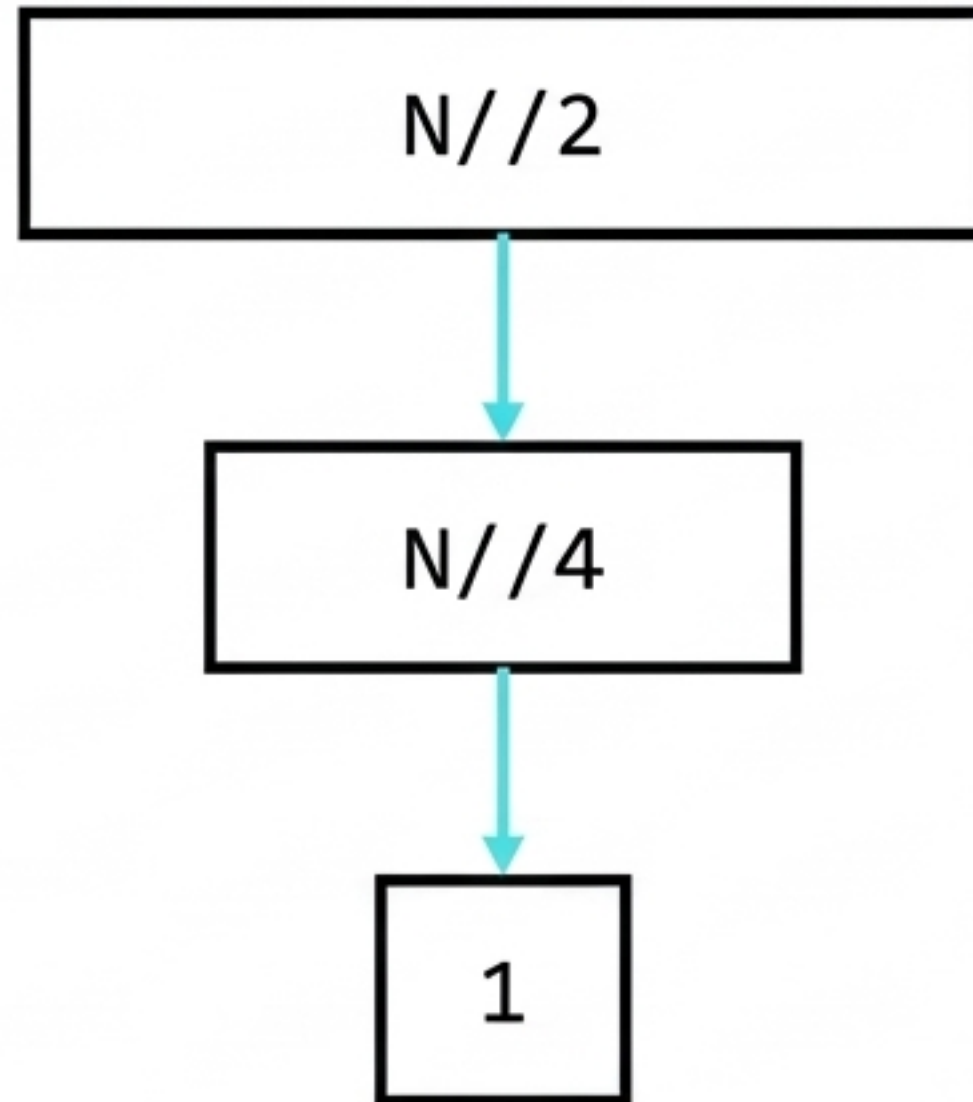
```
def insertion_sort(lista):  
    for i in range(1, len(lista)):  
        clave = lista[i]  
        j = i - 1  
        while j >= 0 and clave < lista[j]:  
            lista[j + 1] = lista[j]  
            j -= 1  
        lista[j + 1] = clave  
    return lista
```

Shell Sort: Concepto

Una mejora de Inserción. Compara e intercambia elementos alejados usando un "Gap" (salto) que se reduce gradualmente.



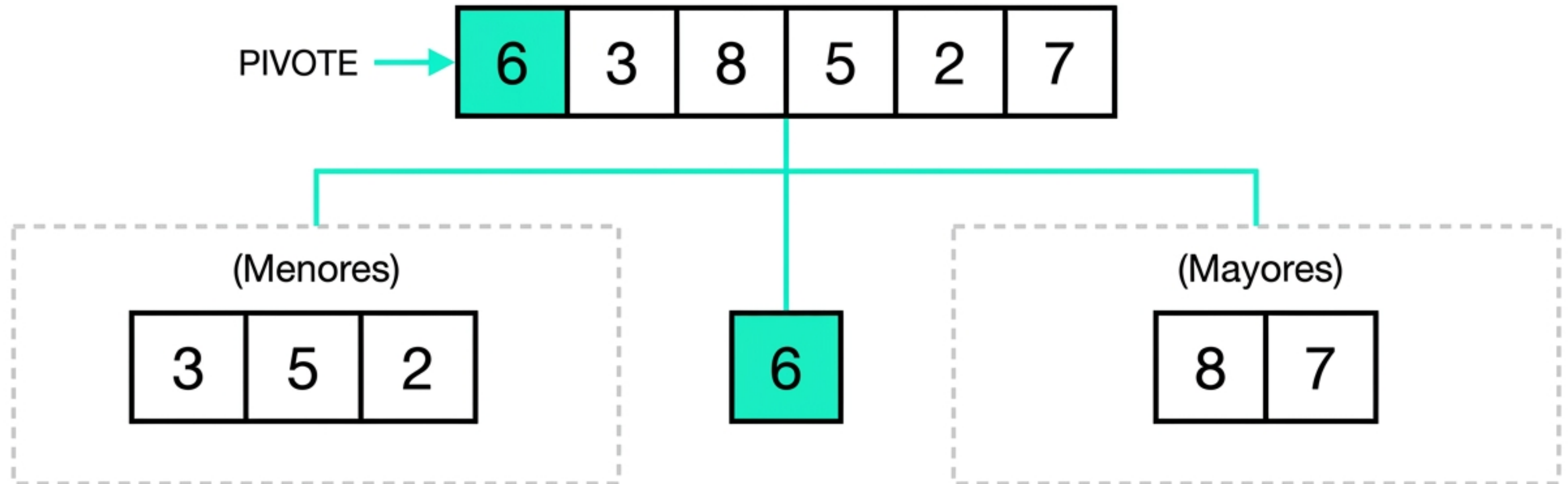
Shell Sort: Proceso y Código



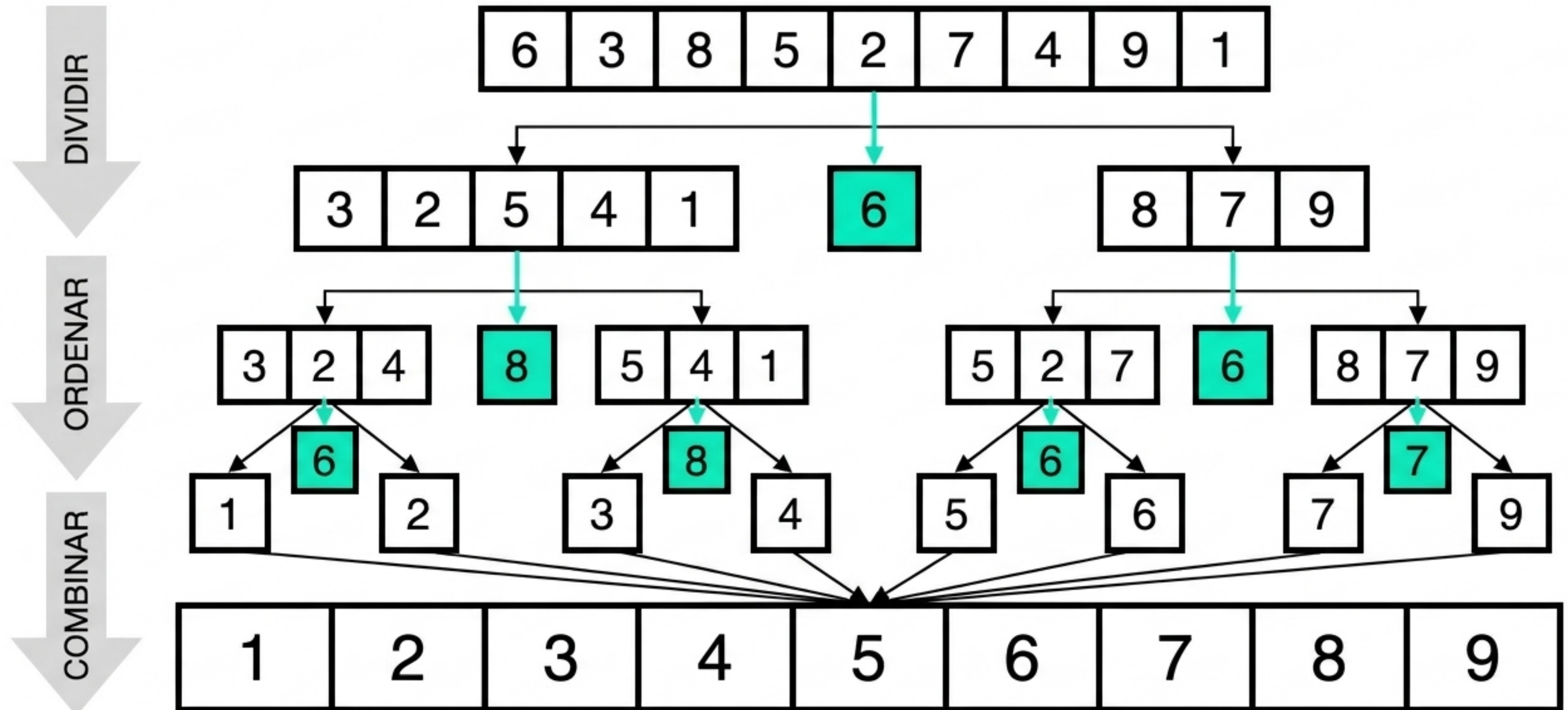
```
def shell_sort(lista):  
    n = len(lista)  
    gap = n // 2  
    while gap > 0:  
        for i in range(gap, n):  
            temp = lista[i]  
            j = i  
            while j >= gap and lista[j - gap] > temp:  
                lista[j] = lista[j - gap]  
                j -= gap  
            lista[j] = temp  
            gap = gap // 2  
    return lista
```

Quick Sort: Concepto

1. Elegir un pivote.
2. Separar menores a la izquierda y mayores a la derecha.
3. Repetir recursivamente.



Quick Sort: Árbol de Particiones



Quick Sort en Python

Promedio: $O(n \log n)$

Peor: $O(n^2)$

```
def quick_sort(lista):  
    if len(lista) <= 1:  
        return lista  
    pivote = lista[len(lista) // 2]  
    menores = [x for x in lista if x < pivote]  
    medios = [x for x in lista if x == pivote]  
    mayores = [x for x in lista if x > pivote]  
    return quick_sort(menores) + medios + quick_sort(mayores)
```

Muy eficiente en promedio y ampliamente utilizado en la industria.

Comparación de Algoritmos

Algoritmo	Mejor Caso	Peor Caso	Uso Práctico
Bubble	$O(n)$	$O(n^2)$	Fácil pero muy lento.
Selection	$O(n^2)$	$O(n^2)$	Simple, pocos intercambios en memoria.
Insertion	$O(n)$	$O(n^2)$	Óptimo para pocos datos o casi ordenados.
Shell	$O(n \log n)$	$O(n^2)$	Mejora de Inserción, paso intermedio.
Quick	$O(n \log n)$	$O(n^2)$	El estándar. Muy rápido en promedio.

¿Cuál elegir?

No existe un único algoritmo perfecto. La elección depende del tamaño, estado inicial y características del sistema.

Datos Pequeños

Insertion

Selection

Bubble

Casi Ordenados

Insertion

Datos Grandes

Quick Sort

¿PREGUNTAS?